

Saath Khata — Planned Features Spec

Purpose: Design doc covering UI/UX, frontend implementation, and backend impact for four

requested features. Each section is self-contained so a developer can pick one up independently.

Table of Contents

1. Quick Login PIN (OTP-style)
2. Staff Delivery Scanner & Payment
3. Add Customer / Vendor with Custom Nickname
4. Bulk Assignment & One-tap Group Charge

1. Quick Login PIN

What it is

After the user logs in once with email + password, the app stores their session (refresh token) securely on the device and prompts them to set a 4-digit PIN. Every subsequent app open shows only the PIN screen — no email, no password. The PIN unlocks the stored session silently in the background.

The backend does not change at all. It just sees a normal token refresh call.

UI / UX Flow

FIRST-EVER LOGIN (UNCHANGED FLOW + NEW PIN SETUP STEP)



you@example.com

Password

.....

[Login as Vendor ▼]

[Log In]

↓ success

Set a Quick Login PIN

You'll use this every time you open the app.

[1][2][3]
[4][5][6]
[7][8][9]
[0]

Enter 4 digits

● ○ ○ ○

↓ confirm PIN

Confirm your PIN

● ● ● ●

(repeat the same 4 digits)

↓ match → saved
→ Dashboard

EVERY LOGIN AFTER THAT

Saath Khata

👋 Welcome back, Ramesh

[1][2][3]
[4][5][6]
[7][8][9]
[0]

● ● ● ●

Use a different account

```
↓ correct PIN
Silent token refresh call
↓ success
Dashboard
```

If PIN is wrong 5 times → show full email + password screen and clear the stored PIN. "Use a different account" clears stored tokens and goes back to full login.

Frontend Implementation

New files needed:

```
lib/features/auth/
  presentation/
    screens/
      pin_setup_screen.dart ← set PIN after first login
      pin_login_screen.dart ← PIN entry on every open
    widgets/
      pin_keypad.dart ← reusable numpad widget
      pin_dots.dart ← 4 dot indicators
  data/
    services/
      pin_service.dart ← encrypt/store/verify PIN
```

Changes to existing files:

- `lib/features/auth/presentation/screens/` — after successful login, check `StorageService.hasPinSet()`. If false → route to `PinSetupScreen`. If true → `Dashboard`.
 - `lib/core/services/storage_service.dart` — add:
 - `savePin(String hashedPin)` - `getPin() → String?` - `clearPin()` - `hasPinSet() → bool`
 - `lib/core/router/app_router.dart` — add a new initial route check:
 - Has refresh token stored AND has PIN set → `PinLoginScreen` - Has refresh token but no PIN → `PinSetupScreen` - Nothing stored → `LoginScreen`
- PIN security:**
- Hash the PIN with SHA-256 before storing (never store raw digits).
 - Store in `flutter_secure_storage` (already used for tokens).
 - On correct PIN entry → call `AuthRepository.refreshTokens(storedRefreshToken)` → update stored tokens.

State: No new BLoC needed. `PinLoginScreen` calls the existing `AuthBloc` with a new event:

```
class AuthPinLoginRequested extends AuthEvent {
  final String pin;
}
```

The bloc handler: verify PIN locally → call refresh endpoint → emit `AuthAuthenticated` .

Backend Impact

None. The PIN login is entirely a Flutter-side concern. The only backend call made is the existing `POST /auth/refresh` with the stored refresh token — exactly the same call the app already makes when the access token expires. The backend has no idea a PIN was involved.

Mock / Dev Mode

For development, hardcode accepted PIN as `1234` :

```
// pin_service.dart
bool verifyPin(String entered) {
  const devPin = '1234';
  if (kDebugMode) return entered == devPin;
  return _hash(entered) == getStoredHash();
}
```

2. Staff Delivery Scanner & Payment

What it is

A staff member can record that they delivered an item to a customer, optionally scan a QR code for the item, enter quantity and price, and with one tap it posts a debit ledger entry to that customer's ledger. No vendor involvement needed after setup.

UI / UX Flow

ENTRY POINT: STAFF DETAIL SCREEN → NEW "RECORD DELIVERY" BUTTON

```
← Rajesh Kumar (Staff)
[ Mark Attendance ]
```

[Advance Salary]
[Record Delivery] ← NEW

RECORD DELIVERY SHEET (BOTTOM SHEET)

Record Delivery

Item Name

Milk

[Scan QR 

Quantity

2

Unit Price

₹ 25

Deliver to

Select customer ▼

← searchable dropdown

Total: ₹ 50

Note (optional)

[Cancel]

[Deliver & Charge]

Tapping **Deliver & Charge**:

↓

✓ Delivered!

Milk × 2 = ₹50

Charged to Sunita Devi

Ledger updated ✓

[Done]

[Record Another]

Frontend Implementation

New files:

```
lib/features/staff/  
  presentation/  
    widgets/  
      record_delivery_sheet.dart    ← the bottom sheet UI  
      delivery_item_scan.dart      ← QR scanner wrapper  
  data/  
    models/  
      delivery_record_model.dart   ← item, qty, price, customerId
```

Changes to existing files:

- [lib/features/staff/presentation/screens/staff_detail_screen.dart](#) —

add "Record Delivery" button that opens `RecordDeliverySheet`.

- [lib/features/staff/presentation/bloc/staff_bloc.dart](#) —

add new event `StaffRecordDelivery` and handler that calls the ledger repository to post the entry.

- [lib/features/ledger/](#) — reuse existing `LedgerRepository.addEntry()`. The

delivery is just a debit ledger entry — no new API endpoint needed. **QR scan:** Add `mobile_scanner` package. On scan, parse QR text as `itemName|unitPrice` (simple pipe-delimited format). If QR doesn't match that format, just pre-fill the item name field with raw text. **Customer dropdown:** Reuse the existing customer list from the vendor's customer list API. Searchable with `DropdownSearch` or a simple `showModalBottomSheet` with a `ListView` and search bar.

Backend Impact

None for the core flow. Posting a delivery = posting a ledger entry. The existing `POST /ledger/entries` endpoint already handles this. Staff acts on behalf of the vendor (using the vendor's JWT stored in the app).

Optional future endpoint:

```
POST /staff/:staffId/deliveries  
{  
  "customerId": "uuid",  
  "itemName": "Milk",  
  "quantity": 2,  
  "unitPrice": 25,  
  "note": "morning delivery"  
}
```

This would create a delivery log separate from the ledger for reporting ("how many deliveries did Rajesh do this month?"). Not needed for MVP — add later.

3. Custom Nickname on Add Customer / Vendor

What it is

When a vendor adds a customer (or links to another vendor), they can give that person a custom display name that only they see. The underlying account name of the other person doesn't matter — the vendor sees their own label everywhere in the app. **Example:** The real account name is `Ramesh Kumar` but you saved them as `Ramesh Doodh Wala`. You see `Ramesh Doodh Wala` everywhere. They see their own name on their side.

UI / UX Flow

UPDATED "ADD CUSTOMER" SHEET

Add Customer

Your label for them ← NEW (required)

Ramesh Doodh Wala

(only you see this name)

Find by phone or email

98765 43210

— OR —

ramesh@example.com

[Add Customer]

If no label is entered, auto-fill with the matched account's real name once the phone/email lookup resolves (can be edited before confirming).

CUSTOMER LIST — SHOWS NICKNAME, NOT ACCOUNT NAME

Customers	
Ramesh Doodh Wala	↗
₹ 450 due	
Sunita Sabji Shop	↗
₹ 0 due	
Ajay Corner Store	↗
₹ 1,200 due	

← your label

Frontend Implementation

Changes to existing files:

- Add `nickname` text field to the Add Customer sheet (wherever it lives in `lib/features/vendor/`).
- In the customer list and customer detail screens, display `link.nickname ?? customer.name` — prefer nickname if set, fall back to real name.
- Pass `nickname` as an extra field in the API call that creates the customer link.

New field on the model:

```
// In customer/link model
final String? nickname; // vendor's private label
```

Backend

Where to store it: The `vendor_customer_links` table (or equivalent) already exists to represent the relationship between a vendor and a customer. Add one column:

```
ALTER TABLE vendor_customer_links
ADD COLUMN nickname VARCHAR(100) NULL;
```

API changes: `POST /vendor/customers` — add optional `nickname` field to request body:

```
{
  "phone": "9876543210",
```

```
"nickname": "Ramesh Doodh Wala"
}
```

`GET /vendor/customers` — include `nickname` in each item of the response. If null, the client falls back to the customer's real `name`. `PATCH /vendor/customers/:customerId` — allow updating nickname separately:

```
{ "nickname": "Ramesh (new label)" }
```

The same pattern applies to vendor-to-vendor links (`vendor_links` table) — add `nickname` there too so vendors can label each other.

4. Bulk Assignment & One-tap Group Charge

What it is

A vendor can define a recurring delivery (e.g. "Daily Milk") with a product name and unit price, assign quantities to multiple customers, and with one tap charge every customer their individual amount to their ledger. The assignment can be saved as a template for daily reuse. This is the "milkman flow" — set up once, run every day in under 10 seconds.

UI / UX Flow

ENTRY POINT: VENDOR DASHBOARD → NEW "BULK DELIVERY" BUTTON

Dashboard

[+ Add Customer]

[Bulk Delivery] ← NEW

[Reports]

STEP 1 — SELECT OR CREATE A TEMPLATE

Bulk Delivery

Your Templates

 Daily Morning Milk →

 Weekly Vegetables →

[+ New Delivery Setup]

STEP 2 — TEMPLATE SETUP (NEW OR EDIT)

← New Delivery Setup

Template Name

Daily Morning Milk

Item / Product

Milk

Unit Price

₹ 25 / litre

[Save & Assign]

STEP 3 — ASSIGN QUANTITIES TO CUSTOMERS

← Daily Morning Milk

₹25/L • Today

Ramesh Doodh Wala

[-] 2 L [+] = ₹50

Sunita Devi

[-] 1 L [+] = ₹25

Ajay Verma

[-] 0 L [+] = ₹0

Priya Gupta

[-] 3 L [+] = ₹75

4 customers · 6 L · ₹150

[Charge All Now]

Customers with qty = 0 are skipped. The summary at the bottom updates live as you tap +/-.

STEP 4 — CONFIRMATION

✓ Done!

3 customers charged
Total collected: ₹150

Ramesh Doodh Wala	₹50	✓
Sunita Devi	₹25	✓
Priya Gupta	₹75	✓

[Back to Templates]

Frontend Implementation

New files:

```
lib/features/bulk_delivery/  
  domain/  
    models/  
      delivery_template.dart      ← id, name, itemName, unitPrice  
      bulk_assignment.dart        ← templateId, date, [{customerId, qty}]  
  data/  
    repositories/  
      bulk_delivery_repository.dart  
      bulk_delivery_repository_impl.dart  
  presentation/  
    bloc/  
      bulk_delivery_cubit.dart    ← state: templates, selected template, assignments  
      bulk_delivery_state.dart  
    screens/  
      bulk_delivery_list_screen.dart ← template picker  
      bulk_template_setup_screen.dart ← create/edit template  
      bulk_assign_screen.dart       ← qty assignment per customer  
      bulk_result_screen.dart       ← success summary
```

How "Charge All Now" works on the frontend: The cubit collects all `{customerId, qty}` pairs where `qty > 0`, calculates `amount = qty × unitPrice` for each, and fires them as parallel ledger entry

requests:

```
await Future.wait(
  assignments
    .where((a) => a.qty > 0)
    .map((a) => ledgerRepo.addEntry(
      customerId: a.customerId,
      amount: a.qty * template.unitPrice,
      note: '${template.itemName} × ${a.qty}',
    )),
);
```

Templates are stored locally (using `shared_preferences` or `isar`) on the device for MVP. They don't need to be on the server yet — a template is just a saved form.

Backend

MVP — no new endpoints needed. "Charge All Now" fires N calls to the existing `POST /ledger/entries` endpoint (one per customer). For small vendor lists (≤ 30 customers) this is fast enough. **Future — batch endpoint (when customer lists grow):**

```
POST /ledger/entries/batch
{
  "entries": [
    { "customerId": "uuid-1", "amount": 50, "note": "Milk × 2" },
    { "customerId": "uuid-2", "amount": 25, "note": "Milk × 1" },
    { "customerId": "uuid-3", "amount": 75, "note": "Milk × 3" }
  ]
}
```

Response:

```
{
  "succeeded": ["uuid-1", "uuid-3"],
  "failed": [
    { "customerId": "uuid-2", "reason": "Customer not linked" }
  ]
}
```

This reduces N HTTP calls to 1 and makes the backend atomic (all succeed or individual failures are reported cleanly). Should be added before shipping to users with > 20 customers. **Template persistence on backend (future):**

```
POST /bulk-templates    ← save a template
GET  /bulk-templates    ← list templates
```

```
PATCH /bulk-templates/:id ← edit
DELETE /bulk-templates/:id ← remove
```

Not needed for MVP since templates are stored locally.

Implementation Order (Recommended)

#	Feature	Effort	Backend Change	Notes
1	Custom Nickname	Small	1 SQL column + 2 API field changes	Lowest risk, high daily value
2	Quick Login PIN	Small	None	Pure Flutter, improves UX immediately
3	Bulk Delivery	Medium	None for MVP	New feature module, reuses ledger API
4	Staff Delivery Scanner	Medium	None for MVP	Reuses ledger + needs camera permission

Shared Dependencies to Add (pubspec.yaml)

Package	Used by
<code>mobile_scanner</code>	Staff delivery QR scan
<code>flutter_secure_storage</code>	Already used — PIN storage
<code>shared_preferences</code> or <code>isar</code>	Bulk delivery templates (local)
<code>dropdown_search</code>	Staff delivery customer picker

Last updated: June 2026